Workspace ONE to Intune — Policy & Compliance Translation

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Workspace ONE to Intune: Policy & Compliance Translation 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Workspace ONE to Intune: Policy & Compliance Translation 5

What this solution is . . . 5

Why the settings layer is the hard part (the why — stated once, here) . . . 5

The three Intune destinations a Workspace ONE policy maps to . . . 5

The per-platform model . . . 5

Workspace ONE internals are deferred to the vendor . . . 6

What you control . . . 6

Naming convention (used for every artifact in this solution) . . . 6

Deployment model . . . 6

Limitations (be honest with buyers) . . . 7

2. Requirements Guide — Workspace ONE to Intune: Policy & Compliance Translation 7

2.1 Workspace ONE policy and compliance export . . . 7

2.2 Intune configuration authoring rights . . . 7

2.3 Intune compliance authoring rights and Conditional Access awareness . . . 8

2.4 Per-platform mapping targets confirmed . . . 8

2.5 The gap register artifact . . . 8

2.6 Policy-scope groups . . . 8

2.7 A parity baseline . . . 9

Checklist . . . 9

3. Implementation Guide — Workspace ONE to Intune: Policy & Compliance Translation 9

Step 1 — Assess: inventory the Workspace ONE source . . . 10

Step 2 — Assess: open the gap register . . . 10

Step 3 — Configure: build the policy-scope groups . . . 10

Step 4 — Configure: map configuration profiles to the settings catalog, per platform . . . 10

Step 5 — Configure: map template-only settings to configuration profiles, per platform . . . 11

Step 6 — Configure: handle unsupported settings and exceptions . . . 11

Step 7 — Configure: map compliance policies, per platform . . . 11

Step 8 — Pilot: apply and confirm on the pilot scope . . . 12

Step 9 — Validate: the parity gate . . . 12

Step 10 — Expand: production . . . 12

5. Testing Guide — Workspace ONE to Intune: Policy & Compliance Translation 13
Test 1 — Source inventory is complete . . . 13
Test 2 — Mapped configuration deployed per platform . . . 13
Test 3 — Mapped compliance deployed per platform . . . 13
Test 4 — Gap register fully decided . . . 13
Test 5 — Parity gate (per platform) . . . 14
Test 6 — Conditional Access actually gates . . . 14
Test 7 — No policy conflicts . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Workspace ONE to Intune: Policy & Compliance Translation 14
A. Parity gate fails — settings reported missing . . . 14
B. Setting maps by name but posture differs . . . 15
C. Compliance policy does not gate access . . . 15
D. Policy conflict on a device . . . 15
E. Device not receiving the mapped policy . . . 15
F. Source-side export issues (Workspace ONE) . . . 15
G. When to escalate . . . 16
7. Best Practices — Workspace ONE to Intune: Policy & Compliance Translation 16
8. Deliverables Summary — Workspace ONE to Intune: Policy & Compliance Translation 17
Documentation . . . 17
PowerShell scripts . . . 17
Named artifacts produced . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Workspace ONE to Intune: Policy & Compliance Translation

The settings are the soul of an MDM migration. Every Workspace ONE configuration profile and every compliance rule exists to enforce a security or management posture, and that posture has to survive the move to Microsoft Intune — or devices quietly become less secure (or simply break) the moment they cut over. Re-creating policy by hand, console-to-console, is where migrations silently lose ground: a passcode rule that does not translate, a Wi-Fi profile that maps to the wrong setting, a compliance check that no longer gates access.

This package gives you a repeatable way to translate Workspace ONE policies and compliance into Intune with proven parity. You inventory the source policy, map it to Intune across every platform in scope, handle what does not translate deliberately rather than by accident, and validate parity before cutover — so nothing weakens silently.

What you get

- A plain-language Solution Overview of the translation model: mapping Workspace ONE profiles and compliance to the Intune settings catalog, configuration profiles, and compliance policy per platform, plus how the parity gate works.
- A Requirements Guide with verify and implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps: inventory the source, map supported settings per platform, handle unsupported settings and exceptions, assign to a pilot, validate parity, then expand.
- Five PowerShell scripts for source-policy discovery, source-versus-Intune parity validation, a translation coverage and gap report, safe remediation of policy conflicts, and a policy-scope group builder.
- Testing and Troubleshooting guides with explicit pass/fail gates.

What this delivers

A mapped, validated Intune policy estate that matches the effective posture of your Workspace ONE configuration, platform by platform — Windows, iOS/iPadOS, Android, and macOS — with every gap recorded and handled, and a parity gate that proves the match on a pilot before you scale.

The deployment model

Assess to Configure to Pilot to Validate to Expand, with a parity gate before any cutover.

Outcome

Every Workspace ONE profile and compliance rule translated into Intune with proven parity — the same effective security posture across platforms, gaps handled on purpose — so nothing silently weakens at cutover, proven on a pilot before you scale.

1. Solution Overview — Workspace ONE to Intune: Policy & Compliance Translation

What this solution is

This is a repeatable way to translate the policy and compliance layer of a Workspace ONE UEM deployment into Microsoft Intune. It is not a device-enrollment migration and it is not an app migration — it is the translation of the settings themselves: every configuration profile and every compliance rule that defines how your devices are secured and managed, moved into Intune so the effective posture is preserved.

The work has four parts: inventory the source policy, map each setting to its Intune equivalent per platform, handle the settings that do not translate, and validate parity before cutover. The output is a set of named, mapped Intune policy objects and a gap register that records every decision.

Why the settings layer is the hard part (the why — stated once, here)

Enrollment moves a device; the policy layer is what makes that device safe. A Workspace ONE configuration profile and a compliance policy each encode intent — a passcode length, an encryption requirement, a Wi-Fi configuration, a jailbreak/root block that gates corporate access. Intune expresses the same intents, but through different objects (the settings catalog, configuration profiles, and compliance policies) with different names, different scopes, and a different per-platform model. Re-typing settings by eye loses fidelity: a setting silently drops, maps to the wrong control, or changes its effective value. When that happens at cutover the device looks managed but is quietly weaker — or a profile conflict breaks a working feature. Translating deliberately, recording every gap, and proving parity on a pilot before you scale is the only way to guarantee the posture you had is the posture you keep. That is the entire reason this solution exists; the rest of the guides simply execute it.

The three Intune destinations a Workspace ONE policy maps to

- Settings catalog — the broadest, most granular surface; the default destination for most

per-platform device configuration. Located at `Devices > Configuration > Create > New policy > (platform) > Settings catalog`.

- Configuration profiles (templates) — Intune's templated profiles (for example Wi-Fi, VPN,

certificate, email, endpoint protection) for settings the settings catalog does not yet cover, at `Devices > Configuration > Create > New policy > (platform) > Templates`.

- Compliance policies — the rules that determine whether a device is compliant (passcode,

encryption, OS version, jailbreak/root, threat level), which Conditional Access then enforces, at `Devices > Compliance > Create policy`.

The per-platform model

Intune policy is created per platform, and a single Workspace ONE profile that spans platforms becomes one Intune object per platform. Every platform in this solution is handled explicitly: Windows, iOS/iPadOS,

Android (Enterprise), and macOS. A setting that exists on the source for one platform may have no equivalent on another — those become entries in the gap register, not silent omissions.

Workspace ONE internals are deferred to the vendor

This solution covers the Intune side with exact navigation. The mechanics of exporting or reading Workspace ONE UEM profiles and compliance policies (console paths, the UEM REST API, the profile XML/payload format) change between Workspace ONE versions and are owned by the vendor — consult the current VMware/Omnissa Workspace ONE UEM documentation for the source-side export. The discovery script flags this and shows where a source export plugs in.

What you control

- Which Workspace ONE settings translate to the settings catalog, to a configuration profile template, or to a compliance policy, per platform.
- How unsupported settings and exceptions are handled (accept, substitute, or defer) — recorded in the gap register.
- The named, reusable Intune objects: the mapped policy sets, the compliance policies, the gap register, and the assignment groups.
- The parity gate: the criteria a mapped policy must meet against the source before cutover.

Naming convention (used for every artifact in this solution)

Every artifact is named `<Purpose> - <Scope> - <Stage>` so objects are self-describing and reusable. The names below are reused verbatim throughout the guides:

- `Mapped Config - Windows - Pilot`, `Mapped Config - iOS - Pilot`, `Mapped Config - Android - Pilot`, `Mapped Config - macOS - Pilot` — the translated configuration policy sets per platform.
- `Mapped Compliance - Windows - Pilot`, `Mapped Compliance - iOS - Pilot`, `Mapped Compliance - Android - Pilot`, `Mapped Compliance - macOS - Pilot` — the translated compliance policies per platform.
- `Policy Translation - All Platforms - Gap Register` — the record of every unsupported setting and exception decision.
- `Policy Scope - Pilot - Stage` and `Policy Scope - Production - Stage` — the Entra security groups that scope assignment by stage.

(At Expand, the `- Pilot` suffix becomes `- Production`; the names are otherwise identical.)

Deployment model

Assess to Configure to Pilot to Validate to Expand, with a parity gate between Validate and Expand: no platform expands to production until its mapped policy passes parity against the source on the pilot.

Limitations (be honest with buyers)

• Workspace ONE source export is vendor-owned and version-specific; this solution flags the export point and defers the source mechanics to VMware/Omnissa documentation.

• Not every Workspace ONE setting has a one-to-one Intune equivalent; some require a settings-catalog substitute, a custom/OMA-URI profile, or are accepted as a gap — each is recorded, not hidden.

• This solution translates policy and compliance; device enrollment, app deployment, and certificate authority migration are separate workstreams (referenced where they intersect).

• Compliance policies only gate access when paired with Conditional Access; configuring Conditional Access is a related but distinct task.

2. Requirements Guide — Workspace ONE to Intune: Policy & Compliance Translation

Verify every requirement before you map anything. Category-wide prerequisites (target tenant, Intune licensing, source access, inventory, enrollment plumbing, pilot group, change control) are in `global-requirements-migrations.md` and are assumed here. The requirements below are specific to the policy and compliance translation work. Each entry: Description · Why · Verify · Implement-if-missing.

2.1 Workspace ONE policy and compliance export

Description: A complete export or readable inventory of the in-scope Workspace ONE UEM configuration profiles and compliance policies, per platform.

Why: You cannot map what you have not captured; the translation begins from a known, complete source list, and parity later is measured against it.

Verify: You can produce, from Workspace ONE UEM, a list of every in-scope profile and compliance policy with its settings and assignments. (Source-side export mechanics are vendor-owned — see the current VMware/Omnissa Workspace ONE UEM documentation.)

Implement-if-missing: Obtain Workspace ONE UEM administrative read access and export the profile and compliance inventory per the vendor documentation; place it where `WS1POL-Discovery.ps1` expects a source export.

2.2 Intune configuration authoring rights

Description: An admin role that can create and assign device configuration policies (settings catalog and templates) in the target tenant.

Why: The mapped policy sets are created in Intune; without authoring rights you cannot build them.

Verify: `https://intune.microsoft.com` loads and you can open `Devices > Configuration > Create`; you hold Intune Administrator or Policy and Profile Manager.

Implement-if-missing: Have a Global Administrator assign Intune Administrator (or the least-privilege Policy and Profile Manager role) to your account.

2.3 Intune compliance authoring rights and Conditional Access awareness

Description: Rights to create compliance policies, and awareness of which Conditional Access policies will consume device-compliance signals.

Why: A translated compliance policy only enforces posture when Conditional Access requires a compliant device; mapping compliance without knowing the gate leaves the rule inert.

Verify: You can open `Devices > Compliance > Create policy`; you can view the Conditional Access policies that require device compliance in `Entra > Protection > Conditional Access`.

Implement-if-missing: Obtain compliance-policy authoring rights; confirm with the identity owner which Conditional Access policies require compliant devices before cutover.

2.4 Per-platform mapping targets confirmed

Description: For each in-scope platform (Windows, iOS/iPadOS, Android Enterprise, macOS), the Intune destination surface is confirmed available (settings catalog, the required template types, compliance policy).

Why: Mapping is per platform; a platform whose enrollment or template prerequisites are not ready cannot receive its translated policy.

Verify: For each platform, `Devices > Configuration > Create > New policy` offers the platform and both Settings catalog and Templates; `Devices > Compliance > Create policy` offers the platform.

Implement-if-missing: Complete the platform enrollment prerequisites (global requirement G5) so the platform's policy surfaces are available; remove any platform that is out of scope from the mapping plan.

2.5 The gap register artifact

Description: A single, named record — `Policy Translation - All Platforms - Gap Register` — that captures every source setting with no clean Intune equivalent and the decision made for it.

Why: Unsupported settings handled silently are exactly how posture is lost; the register makes every gap an explicit, reviewable decision.

Verify: The gap register exists (a spreadsheet or document) with columns for source setting, platform, Intune destination or "none", decision (accept/substitute/defer), and owner.

Implement-if-missing: Create `Policy Translation - All Platforms - Gap Register` from the template in the Implementation Guide before mapping begins.

2.6 Policy-scope groups

Description: Microsoft Entra security groups that scope policy assignment by stage — `Policy Scope - Pilot - Stage` first, `Policy Scope - Production - Stage` at expand.

Why: Translated policy must land on a controlled pilot before the fleet; stage-scoped groups make the Pilot-then-Expand model concrete and reversible.

Verify: `Entra > Groups`; `Policy Scope - Pilot - Stage` exists and contains representative devices across the in-scope platforms.

Implement-if-missing: Create the groups with `WS1POL-Set-PolicyScopeTargeting.ps1`, or manually at `Entra > Groups > New group > Security`.

2.7 A parity baseline

Description: An agreed definition of what "parity" means for this migration — the setting-level criteria a mapped Intune policy must meet against its Workspace ONE source to pass the gate.

Why: The parity gate at Validate needs an objective pass/fail definition, not a judgment call.

Verify: A written parity definition exists (for example: every in-scope source setting is mapped, substituted, or registered as a gap; effective values match; compliance rules gate the same access).

Implement-if-missing: Define parity with the security owner before mapping; record it alongside the gap register.

Checklist

- Workspace ONE profile + compliance export captured (vendor docs)
- Intune configuration authoring rights
- Compliance authoring rights + Conditional Access consumers identified
- Per-platform mapping targets available (Windows / iOS / Android / macOS)
- `Policy Translation - All Platforms - Gap Register` created
- `Policy Scope - Pilot - Stage` group created with representative devices
- Parity baseline defined and agreed

3. Implementation Guide — Workspace ONE to Intune: Policy & Compliance Translation

Follow these steps in order. The Intune work is done in the Microsoft Intune admin center (`https://intune.microsoft.com`). The Workspace ONE source export is vendor-owned and is flagged at Step 1 — perform it per the current VMware/Omnissa Workspace ONE UEM documentation.

Model: Assess (inventory the source) to Configure (map per platform, handle gaps) to Pilot (assign to the pilot scope) to Validate (parity gate) to Expand (production). No platform passes Validate to Expand until its mapped policy clears the parity gate.

The artifact names from the Overview are reused verbatim below.

Step 1 — Assess: inventory the Workspace ONE source

1. Export the in-scope Workspace ONE UEM configuration profiles and compliance policies per platform.

The console paths, UEM REST API, and payload format are vendor-owned and version-specific — follow the current VMware/Omnissa Workspace ONE UEM documentation. (Deferred to vendor docs.)

1. Save the export where `WS1POL-Discovery.ps1` expects it, and run `WS1POL-Discovery.ps1` to produce

a normalized source inventory list (profiles and compliance rules by platform). This script is read-only.

1. Confirm the inventory is complete against the source console before proceeding — completeness here

is what parity is later measured against.

Step 2 — Assess: open the gap register

1. Create `Policy Translation - All Platforms - Gap Register` with one row per source setting and

these columns: Source setting · Platform · Intune destination (settings catalog / template / compliance / none) · Decision (map / substitute / defer / accept-gap) · Effective-value note · Owner.

1. Pre-populate it from the Step 1 inventory; every source setting gets a row before any mapping

begins.

Step 3 — Configure: build the policy-scope groups

1. Run `WS1POL-Set-PolicyScopeTargeting.ps1` to create `Policy Scope - Pilot - Stage` (and

`Policy Scope - Production - Stage` for later), or create them manually at `Entra > Groups > New group > Security`.

1. Add a representative device for each in-scope platform to `Policy Scope - Pilot - Stage`.

Step 4 — Configure: map configuration profiles to the settings catalog, per platform

Do this once per in-scope platform. The example is Windows; repeat for iOS/iPadOS, Android, macOS.

1. Go to `Devices > Configuration > Create > New policy`.

2. Platform: select the platform (for example Windows 10 and later). Profile type: Settings catalog.

Select Create.

1. Name: `Mapped Config - Windows - Pilot` (use the matching platform name for each platform:

`Mapped Config - iOS - Pilot`, `Mapped Config - Android - Pilot`, `Mapped Config - macOS - Pilot`). Select Next.

1. Select Add settings, then search the settings picker for each source setting's equivalent and add it, setting the same effective value as the Workspace ONE source. For each setting, mark its row in the gap register as map and note the effective value.
1. For any source setting with no settings-catalog equivalent, leave it for Step 5 (template) or Step 6 (gap) — do not approximate it silently.
1. Assignments: add `Policy Scope - Pilot - Stage`. Select Next, then Create.

Step 5 — Configure: map template-only settings to configuration profiles, per platform

For source settings that the settings catalog does not cover (commonly Wi-Fi, VPN, certificates, email, and some endpoint-protection settings):

1. Go to `Devices > Configuration > Create > New policy`.
2. Platform: the platform. Profile type: Templates, then select the matching template (for example Wi-Fi, VPN, or SCEP/PKCS certificate). Select Create.
1. Name it `Mapped Config - <Platform> - Pilot` with a template suffix that identifies it (for example `Mapped Config - Windows - Pilot (Wi-Fi)`), so it remains self-describing.
1. Enter the settings to match the source profile's effective values. Mark each mapped row in the gap register.
1. Assignments: add `Policy Scope - Pilot - Stage`. Select Next, then Create.

Step 6 — Configure: handle unsupported settings and exceptions

For every gap-register row still unresolved:

1. Decide and record one of: substitute (a different Intune setting achieves the same effective posture — note it), defer (handled by a separate workstream such as certificate-authority or app migration — note the owner), or accept-gap (no equivalent and the risk is accepted — note who accepted it).
1. For settings that need a control Intune exposes only via custom configuration, use a custom / OMA-URI profile where the platform supports it, and record the OMA-URI in the register. Do not invent settings that do not exist.
1. The gap register, fully decided, is a deliverable in its own right — no source setting may remain undecided at the parity gate.

Step 7 — Configure: map compliance policies, per platform

Do this once per in-scope platform.

1. Go to `Devices > Compliance > Create policy`.
2. Platform: select the platform. Select Create.

3. Name: `Mapped Compliance - Windows - Pilot` (and the matching platform name for each: `Mapped Compliance - iOS - Pilot`, `Mapped Compliance - Android - Pilot`, `Mapped Compliance - macOS - Pilot`).

1. Compliance settings: set each rule to match the Workspace ONE compliance policy's intent — for example minimum OS version, passcode/password requirements, encryption required, jailbreak/root (rooted device) block, and threat level where a Mobile Threat Defense connector is present. Record each mapped rule in the gap register.

1. Actions for noncompliance: set the same effective consequence as the source (for example mark noncompliant immediately or after a grace period).

1. Assignments: add `Policy Scope - Pilot - Stage`. Select Next, then Create.

2. Confirm with the identity owner that the Conditional Access policy requiring a compliant device covers these devices, so the compliance signal actually gates access.

Step 8 — Pilot: apply and confirm on the pilot scope

1. Trigger a sync on a pilot device per platform (the device-side sync path differs per platform).

2. Run `WS1POL-Report.ps1` to confirm each `Mapped Config - <Platform> - Pilot` and `Mapped Compliance - <Platform> - Pilot` policy is deployed and shows assigned devices. This script is read-only.

Step 9 — Validate: the parity gate

1. Run `WS1POL-Validation.ps1` to compare the source inventory (Step 1) against the deployed Intune policy, per platform, and produce a parity result: mapped, substituted, deferred, accepted-gap, or missing. This script is read-only.

1. Run `WS1POL-Report.ps1` for per-device compliance and configuration state on the pilot.

2. The parity gate passes for a platform when: every in-scope source setting is mapped, substituted, deferred, or accepted-gap (none missing); effective values match; and the compliance policy gates the same access. If anything is missing, resolve it in the gap register and re-map.

1. If a policy conflict is detected (a setting reported in conflict on a device), run `WS1POL-Remediation.ps1` to identify and, with confirmation, resolve the conflicting assignment. It defaults to `-WhatIf`.

Step 10 — Expand: production

1. Once a platform clears the parity gate, change its mapped configuration and compliance policy assignments from `Policy Scope - Pilot - Stage` to `Policy Scope - Production - Stage`, or duplicate each policy to a `- Production` copy and assign the production group.

1. Add the fleet's devices to `Policy Scope - Production - Stage` in waves per the wave plan (global

requirement G6).

1. Re-run `WS1POL-Validation.ps1` and `WS1POL-Report.ps1` after each wave to confirm parity holds and

no device fell out of compliance.

Order of operations

`Source inventory > Gap register > Scope groups > Map config (catalog) > Map config (templates) > Handle gaps > Map compliance > Apply on pilot > Parity gate > Expand`

5. Testing Guide — Workspace ONE to Intune: Policy & Compliance Translation

Run these after the mapped policies apply to the pilot (Implementation Step 8) and again after each expansion wave. The parity gate (Test 5) is the go/no-go for cutover.

Test 1 — Source inventory is complete

Do: Run `WS1POL-Discovery.ps1` against the Workspace ONE source export and compare the normalized list to the source console.

Expected: Every in-scope configuration profile and compliance policy, per platform, appears in the inventory. No source object is missing.

Test 2 — Mapped configuration deployed per platform

Do: In Intune, open `Devices > Configuration` and check each `Mapped Config - <Platform> - Pilot` policy's Device status; or run `WS1POL-Report.ps1`.

Expected: Each platform's mapped configuration policy shows Succeeded on the pilot devices.

Test 3 — Mapped compliance deployed per platform

Do: In Intune, open `Devices > Compliance`, check each `Mapped Compliance - <Platform> - Pilot` policy's Device status; confirm pilot devices evaluate as expected.

Expected: Pilot devices report compliant under the translated rules, matching the source policy's intent.

Test 4 — Gap register fully decided

Do: Review `Policy Translation - All Platforms - Gap Register`.

Expected: Every source setting row has a decision (map / substitute / defer / accept-gap) with an owner. No row is undecided.

Test 5 — Parity gate (per platform)

Do: Run `WS1POL-Validation.ps1` to compare source inventory against deployed Intune policy per platform.

Expected: For the platform under test, no setting is reported missing; effective values match; the compliance policy gates the same access. This is the cutover gate.

Test 6 — Conditional Access actually gates

Do: With a pilot device deliberately made noncompliant against a translated rule, attempt a Conditional-Access-protected resource.

Expected: Access is blocked while noncompliant and restored when compliant — the compliance signal reaches Conditional Access.

Test 7 — No policy conflicts

Do: Run `WS1POL-Report.ps1` (per-device configuration state) and review for settings in conflict.

Expected: No setting reports a conflict on pilot devices. If any does, `WS1POL-Remediation.ps1` (run with `-WhatIf` first) identifies the conflicting assignment.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 4 pass; Test 5 (parity) passes for the platform; Test 6 confirms gating
Expand pass	Tests 2, 3, 5, 7 pass for each wave on `Policy Scope - Production - Stage`

6. Troubleshooting Guide — Workspace ONE to Intune: Policy & Compliance Translation

Use `WS1POL-Discovery.ps1` for the source inventory, `WS1POL-Validation.ps1` for the parity check, `WS1POL-Report.ps1` for per-device state, and `WS1POL-Remediation.ps1` (defaults to `-WhatIf`) for conflicting assignments.

A. Parity gate fails — settings reported missing

Symptom	Cause	Remediation
Validation lists a source setting as missing	It was not mapped, or mapped on the wrong platform	Find the gap-register row; map it in the correct `Mapped Config - <Platform> - Pilot`, or record substitute/defer/accept-gap

Symptom	Cause	Remediation
A whole platform shows many missing	The platform's policy was not assigned to the pilot scope	Confirm assignment of `Policy Scope - Pilot - Stage`; re-run `WS1POL-Report.ps1`
Source export incomplete	Inventory captured only part of the source	Re-export per VMware/Omnissa documentation; re-run `WS1POL-Discovery.ps1`

B. Setting maps by name but posture differs

- A control mapped with a different effective value silently changes posture. Compare the effective value, not just the setting name, against the source; correct the value in the mapped policy and re-run `WS1POL-Validation.ps1`.

C. Compliance policy does not gate access

Symptom	Cause	Remediation
Noncompliant device still reaches resources	No Conditional Access policy requires a compliant device	Confirm with the identity owner that Conditional Access requires device compliance for these users/devices
Device shows compliant but should not	A translated rule is missing or weaker than the source	Compare the rule set in `Mapped Compliance - <Platform> - Pilot` to the source; tighten to match

D. Policy conflict on a device

- A setting reported in conflict means two policies set it differently. Run `WS1POL-Remediation.ps1` with `-WhatIf` to identify the conflicting assignment, then re-run without `-WhatIf` (with confirmation) to resolve it. Conflicts commonly come from a leftover Intune policy or two mapped policies overlapping — consolidate.

E. Device not receiving the mapped policy

- Confirm the device is enrolled and a member of `Policy Scope - Pilot - Stage`; trigger a sync (device-side path differs per platform); then re-run `WS1POL-Report.ps1`. Coexistence note: until a platform clears parity, Workspace ONE policy may still be authoritative on the device — confirm which MDM the device is enrolled in.

F. Source-side export issues (Workspace ONE)

- Export mechanics (console paths, the UEM REST API, payload format) are vendor-owned and

version-specific. If the export is incomplete or malformed, consult the current VMware/Omnissa Workspace ONE UEM documentation — this solution defers all source-side mechanics there.

G. When to escalate

If parity validation is clean, the policy is assigned, the device is synced and enrolled in Intune, and a setting still does not apply or a compliance rule still does not gate, collect the `WS1POL-Validation.ps1` and `WS1POL-Report.ps1` output and open a Microsoft support case for the Intune side (or a VMware/Omnissa case for a source-export discrepancy).

7. Best Practices — Workspace ONE to Intune: Policy & Compliance Translation

• Inventory before you map. A complete, normalized source inventory is what parity is measured against; map nothing until the source list is verified complete with `WS1POL-Discovery.ps1`.

• Make every gap an explicit decision. The single most important practice: no source setting is dropped silently. Every setting with no clean equivalent gets a row in `Policy Translation - All Platforms - Gap Register` with a decision and an owner.

• Translate effective values, not just setting names. A setting that maps by name but with a different effective value silently changes posture. Confirm the value, not only the control.

• Work platform by platform. Intune policy is per platform; a profile that spanned platforms in Workspace ONE becomes one Intune object per platform. Handle Windows, iOS/iPadOS, Android, and macOS each on their own pass.

• Prefer the settings catalog, fall back to templates. Map most configuration in the settings catalog for granularity; use template profiles only where the catalog does not cover the setting (Wi-Fi, VPN, certificates, email).

• Pin compliance to Conditional Access. A translated compliance policy only protects anything when Conditional Access requires a compliant device; confirm the gate exists before cutover.

• Name everything `<Purpose> - <Scope> - <Stage>`. Reuse the names verbatim so every object is self-describing and the Pilot-to-Production promotion is mechanical.

• Gate on parity, not on schedule. No platform expands to production until it clears the parity gate on the pilot; the gate, not the calendar, decides cutover.

• Keep coexistence and rollback ready. Until a platform clears parity, the Workspace ONE policy remains authoritative; the rollback is to revert the Intune assignment (global requirement G7).

• Re-validate after every wave. Run `WS1POL-Validation.ps1` and `WS1POL-Report.ps1` after each expansion wave so parity is proven to hold, not assumed.

• Resolve conflicts deliberately. Use `WS1POL-Remediation.ps1` with `-WhatIf` first; never blind-fix a conflicting assignment.

8. Deliverables Summary — Workspace ONE to Intune: Policy & Compliance Translation

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	The translation model, the three Intune destinations, the per-platform model, naming, and the parity gate (the why, stated once)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Migrations)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: inventory, map per platform, handle gaps, map compliance, pilot, parity gate, expand
5. Testing Guide	Validation procedures with explicit pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`WS1POL-Discovery.ps1`	Normalizes the Workspace ONE source export into a per-platform profile + compliance inventory (read-only; source export deferred to vendor docs)
`WS1POL-Validation.ps1`	Parity check — compares source inventory against deployed Intune configuration and compliance policy per platform (read-only, Microsoft Graph)
`WS1POL-Report.ps1`	Per-device and per-policy configuration + compliance state report (read-only, Microsoft Graph)
`WS1POL-Remediation.ps1`	Identifies and, with confirmation, resolves conflicting policy assignments; defaults to `-WhatIf`
`WS1POL-Set-PolicyScopeTargeting.ps1`	Creates and populates the `Policy Scope - Pilot/Production - Stage` Entra groups

Named artifacts produced

- `Mapped Config - <Platform> - Pilot` and `- Production` (Windows / iOS / Android / macOS)
- `Mapped Compliance - <Platform> - Pilot` and `- Production` (Windows / iOS / Android / macOS)
- `Policy Translation - All Platforms - Gap Register`
- `Policy Scope - Pilot - Stage` and `Policy Scope - Production - Stage`

What the buyer receives

- This guide set as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess to Configure to Pilot to Validate to Expand, with a parity gate before cutover.

Outcome

Every Workspace ONE profile and compliance rule translated into Intune with proven parity — the same effective posture across platforms, every gap handled deliberately — validated on a pilot before the fleet scales.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WS1POL-Discovery.ps1

Purpose: Reads a Workspace ONE UEM export (configuration profiles + compliance policies) and produces a normalized, per-platform inventory list that the mapping and parity-validation steps measure against. The Workspace ONE export itself is vendor-owned and version-specific: the console paths, UEM REST API, and payload format are documented by VMware/Omnissa, not here. This script consumes an export you have already produced (JSON or CSV) and normalizes it. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1POL-Discovery.ps1 -ExportPath C:\Temp\ws1-export.json -CsvPath C:\Temp\ws1-inventory.csv -ExportPath : a Workspace ONE export file (JSON array or CSV). DEFERRED: produce it per the current VMware/Omnissa Workspace ONE UEM documentation. EXPECTED RESULTS A per-platform count of profiles and compliance policies, and (optional) a normalized CSV with one row per source setting: Platform, Type (Profile/Compliance), Name, Setting. .NOTES Workspace ONE to Intune migration — read-only. Source export deferred to VMware/Omnissa docs. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ExportPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1POL-Discovery.ps1 -ExportPath C:\Temp\ws1-export.json -CsvPath C:\Temp\ws1-inventory.csv -ExportPath : a Workspace ONE export file (JSON array or CSV). DEFERRED: produce it per the current VMware/Omnissa Workspace ONE UEM documentation. EXPECTED RESULTS A per-platform count of profiles and compliance policies, and (optional) a normalized CSV with one row per source setting: Platform, Type (Profile/Compliance), Name, Setting. .NOTES Workspace ONE to Intune migration — read-only. Source export deferred to VMware/Omnissa docs. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ExportPath, [string]$CsvPath)

WS1POL-Remediation.ps1

Purpose: Identifies devices reporting a configuration setting in conflict (the same setting set by more than one policy) and, when run without -WhatIf, removes the conflicting assignment you select from the policy you specify — the safe fix for the overlap that breaks parity. Defaults to -WhatIf so nothing changes until you confirm. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\WS1POL-Remediation.ps1 Apply (remove a conflicting group assignment from a policy): powershell -ExecutionPolicy Bypass -File .\WS1POL-Remediation.ps1 -PolicyId -RemoveGroupId -WhatIf:$false Requires Microsoft Graph SDK with DeviceManagementConfiguration.ReadWrite.All. EXPECTED RESULTS -WhatIf (default): a list of configuration policies in a conflicting state and the requested assignment change, with NO change made. Applied: the named group assignment is removed from the named policy; re-run WS1POL-Validation.ps1. .NOTES Workspace ONE to Intune migration — defaults to -WhatIf. Read-only unless -WhatIf:$false is set. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([string]$PolicyId, [string]$RemoveGroupId) # Default to -WhatIf unless the caller explicitly disabled it. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\WS1POL-Remediation.ps1 Apply (remove a conflicting group assignment from a policy): powershell -ExecutionPolicy Bypass -File .\WS1POL-Remediation.ps1 -PolicyId <id> -RemoveGroupId <groupId> -WhatIf:$false Requires Microsoft Graph SDK with DeviceManagementConfiguration.ReadWrite.All.

EXPECTED RESULTS -WhatIf (default): a list of configuration policies in a conflicting state and the requested assignment change, with NO change made. Applied: the named group assignment is removed from the named policy; re-run WS1POL-Validation.ps1. .NOTES Workspace ONE to Intune migration — defaults to -WhatIf. Read-only unless -WhatIf:$false is set. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([string]$PolicyId, [string]$RemoveGroupId) # Default to -WhatIf unless the caller explicitly disabled it. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

WS1POL-Report.ps1

Purpose: Reports the deployment state of the mapped Intune policies: which 'Mapped Config' and 'Mapped Compliance' policies exist, their assigned groups, and the device compliance state for the managed fleet — so you can confirm the translated policy reached the pilot/production scope and devices are compliant under the translated rules. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1POL-Report.ps1 -CsvPath C:\Temp\ws1pol-devices.csv Requires Microsoft Graph SDK with DeviceManagementConfiguration.Read.All and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A list of the mapped configuration and compliance policies with assigned group IDs, plus a per-device compliance summary and (optional) a per-device CSV. .NOTES Workspace ONE to Intune migration — read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1POL-Report.ps1 -CsvPath C:\Temp\ws1pol-devices.csv Requires Microsoft Graph SDK with DeviceManagementConfiguration.Read.All and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A list of the mapped configuration and compliance policies with assigned group IDs, plus a per-device compliance summary and (optional) a per-device CSV. .NOTES Workspace ONE to Intune migration — read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

WS1POL-Set-PolicyScopeTargeting.ps1

Purpose: Creates the Microsoft Entra security groups that scope mapped-policy assignment by stage — 'Policy Scope - Pilot - Stage' and 'Policy Scope - Production - Stage' — and optionally adds device object IDs to the pilot group, so the Pilot-then-Expand model is concrete and reversible. Idempotent: existing groups are reused, not duplicated. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1POL-Set-PolicyScopeTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\WS1POL-Set-PolicyScopeTargeting.ps1 -PilotDeviceIds id1,id2 Requires Microsoft Graph SDK with Group.ReadWrite.All (and GroupMember.ReadWrite.All to add members). EXPECTED RESULTS Both policy-scope groups exist (created if missing); any supplied device object IDs are added to 'Policy Scope - Pilot - Stage'. Prints each group's object ID for use in policy assignment. .NOTES Workspace ONE to Intune migration — creates/updates Entra groups (write). Membership add is opt-in. #> [CmdletBinding(SupportsShouldProcess)] param([string[]]$PilotDeviceIds)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1POL-Set-PolicyScopeTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\WS1POL-Set-PolicyScopeTargeting.ps1 -PilotDeviceIds id1,id2 Requires Microsoft Graph SDK with Group.ReadWrite.All (and GroupMember.ReadWrite.All to add members). EXPECTED RESULTS Both policy-scope groups exist (created if missing); any supplied device object IDs are added to 'Policy Scope - Pilot - Stage'. Prints each group's object ID for use in policy assignment. .NOTES Workspace ONE to Intune migration — creates/updates Entra groups (write). Membership add is opt-in. #> [CmdletBinding(SupportsShouldProcess)] param([string[]]$PilotDeviceIds)

WS1POL-Validation.ps1

Purpose: Reads the normalized source inventory (from WS1POL-Discovery.ps1) and the deployed Intune configuration + compliance policies, then reports parity per platform: which mapped policies exist in Intune for each platform and the source-versus-target setting counts. This is the data behind the parity gate; the human gap register records map/substitute/defer/accept-gap decisions that a tool cannot infer. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1POL-Validation.ps1 -InventoryCsv C:\Temp\ws1-inventory.csv Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS Per-platform: source setting count, the matching 'Mapped Config - ' settings-catalog policy and its setting count, and the 'Mapped Compliance - ' policy. A platform with no matching mapped policy is flagged for follow-up. .NOTES Workspace ONE to Intune migration — read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1POL-Validation.ps1 -InventoryCsv C:\Temp\ws1-inventory.csv Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS Per-platform: source setting count, the matching 'Mapped Config - <Platform>' settings-catalog policy and its setting count, and the 'Mapped Compliance - <Platform>' policy. A platform with no matching mapped policy is flagged for follow-up. .NOTES Workspace ONE to Intune migration — read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv)